二维几何

库实数类实现（双精度）

```
using Real = int;
using Point = complex<Real>;

Real cross(const Point &a, const Point &b) {
    return (conj(a) * b).imag();
}
Real dot(const Point &a, const Point &b) {
    return (conj(a) * b).real();
}
```

平面几何必要初始化

字符串读入浮点数

```
const int Knum = 4;
int read(int k = Knum) {
    string s;
    cin >> s;

    int num = 0;
    int it = s.find('.');
    if (it != -1) { // 存在小数点
        num = s.size() - it - 1; // 计算小数位数
        s.erase(s.begin() + it); // 删除小数点
    }
    for (int i = 1; i <= k - num; i++) { // 补全小数位数
        s += '0';
    }
    return stoi(s);
}
```

预置函数

```
using ld = long double;
const ld PI = acos(-1);
const ld EPS = 1e-7;
const ld INF = numeric_limits<ld>::max();
#define cc(x) cout << fixed << setprecision(x);

ld fgcd(ld x, ld y) { // 实数域gcd
    return abs(y) < EPS ? abs(x) : fgcd(y, fmod(x, y));
}
template<class T, class S> bool equal(T x, S y) {
    return -EPS < x - y && x - y < EPS;
}
```

```cpp
template<class T> int sign(T x) {
    if (-EPS < x && x < EPS) return 0;
    return x < 0 ? -1 : 1;
}
template<class T> T sqr(T x) {
    return x * x;
}

template<class T> T disEx(Pt a1,Pt a2)
{
    return (a1.x-a2.x)*(a1.x-a2.x)+(a1.y-a2.y)*(a1.y-a2.y);
}
```

点线封装

```cpp
template<class T> struct Point { // 在C++17下使用 emplace_back 绑定可能会导致CE!
    T x, y;
    Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {} // 初始化
    template<class U> operator Point<U>() { // 自动类型匹配
        return Point<U>(U(x), U(y));
    }
    Point &operator+=(Point p) & { return x += p.x, y += p.y, *this; }
    Point &operator+=(T t) & { return x += t, y += t, *this; }
    Point &operator-=(Point p) & { return x -= p.x, y -= p.y, *this; }
    Point &operator-=(T t) & { return x -= t, y -= t, *this; }
    Point &operator*=(T t) & { return x *= t, y *= t, *this; }
    Point &operator/=(T t) & { return x /= t, y /= t, *this; }
    Point operator-() const { return Point(-x, -y); }
    friend Point operator+(Point a, Point b) { return a += b; }
    friend Point operator+(Point a, T b) { return a += b; }
    friend Point operator-(Point a, Point b) { return a -= b; }
    friend Point operator-(Point a, T b) { return a -= b; }
    friend Point operator*(Point a, T b) { return a *= b; }
    friend Point operator*(T a, Point b) { return b *= a; }
    friend Point operator/(Point a, T b) { return a /= b; }
    friend bool operator<(Point a, Point b) {
        return equal(a.x, b.x) ? a.y < b.y - EPS : a.x < b.x - EPS;
    }
    friend bool operator>(Point a, Point b) { return b < a; }
    friend bool operator==(Point a, Point b) { return !(a < b) && !(b < a); }
    friend bool operator!=(Point a, Point b) { return a < b || b < a; }
    friend auto &operator>>(istream &is, Point &p) {
        return is >> p.x >> p.y;
    }
    friend auto &operator<<(ostream &os, Point p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};
template<class T> struct Line {
    Point<T> a, b;
    Line(Point<T> a_ = Point<T>(), Point<T> b_ = Point<T>()) : a(a_), b(b_) {}
```

```
    template<class U> operator Line<U>() { // 自动类型匹配
        return Line<U>(Point<U>(a), Point<U>(b));
    }
    friend auto &operator<<(ostream &os, Line l) {
        return os << "<" << l.a << ", " << l.b << ">";
    }
};
```

叉乘

定义公式 $a \times b = |a||b| \sin \theta$。

```
template<class T> T cross(Point<T> a, Point<T> b) { // 叉乘
    return a.x * b.y - a.y * b.x;
}
template<class T> T cross(Point<T> p1, Point<T> p2, Point<T> p0) { // 叉乘 (p1 - p0)
x (p2 - p0);
    return cross(p1 - p0, p2 - p0);
}
```

点乘

定义公式 $a \times b = |a||b| \cos \theta$。

```
template<class T> T dot(Point<T> a, Point<T> b) { // 点乘
    return a.x * b.x + a.y * b.y;
}
template<class T> T dot(Point<T> p1, Point<T> p2, Point<T> p0) { // 点乘 (p1 - p0) *
(p2 - p0);
    return dot(p1 - p0, p2 - p0);
}
```

欧几里得距离公式

最常用的距离公式。**需要注意**，开根号会丢失精度，如无强制要求，先不要开根号，留到最后一步一起开。

```
template <class T> ld dis(T x1, T y1, T x2, T y2) {
    ld val = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
    return sqrt(val);
}
template <class T> ld dis(Point<T> a, Point<T> b) {
    return dis(a.x, a.y, b.x, b.y);
}
```

曼哈顿距离公式

```
template <class T> T dis1(Point<T> p1, Point<T> p2) { // 曼哈顿距离公式
    return abs(p1.x - p2.x) + abs(p1.y - p2.y);
}
```

将向量转换为单位向量

```
Point<ld> standardize(Point<ld> vec) { // 转换为单位向量
    return vec / sqrt(vec.x * vec.x + vec.y * vec.y);
}
```

向量旋转

将当前向量移动至原点后顺时针旋转 90°，即获取垂直于当前向量的、起点为原点的向量。在计算垂线时非常有用。例如，要想获取点 a 绕点 o 顺时针旋转 90° 后的点，可以这样书写代码：`auto ans = o + rotate(o, a);`；如果是逆时针旋转，那么只需更改符号即可：`auto ans = o - rotate(o, a);`。

```
template<class T> Point<T> rotate(Point<T> p1, Point<T> p2) { // 旋转
    Point<T> vec = p1 - p2;
    return {-vec.y, vec.x};
}
```

平面角度与弧度

弧度角度相互转换

```
ld toDeg(ld x) { // 弧度转角度
    return x * 180 / PI;
}
ld toArc(ld x) { // 角度转弧度
    return PI / 180 * x;
}
```

正弦定理

$\frac{a}{\sin A}=\frac{b}{\sin B}=\frac{c}{\sin C}=2R$，其中 R 为三角形外接圆半径；

余弦定理（已知三角形三边，求角）

$\cos C=\frac{a^2+b^2-c^2}{2ab},\cos B=\frac{a^2+c^2-b^2}{2ac},\cos A=\frac{b^2+c^2-a^2}{2bc}$。可以借此推导出三角形面积公式 $S_{\triangle ABC}=\frac{ab\cdot\sin C}{2}=\frac{bc\cdot\sin A}{2}=\frac{ac\cdot\sin B}{2}$。

注意，计算格式是：由 b,c,a 三边求 $\angle A$；由 a,c,b 三边求 $\angle B$；由 a,b,c 三边求 $\angle C$。

```
ld angle(ld a, ld b, ld c) { // 余弦定理
    ld val = acos((a * a + b * b - c * c) / (2.0 * a * b)); // 计算弧度
    return val;
}
```

求两向量的夹角

能够计算 $[0°, 180°]$ 区间的角度。

```
ld angle(Point<ld> a, Point<ld> b) {
    ld val = abs(cross(a, b));
    return abs(atan2(val, a.x * b.x + a.y * b.y));
}
```

向量旋转任意角度

逆时针旋转，转换公式：$\begin{cases} x' = x\cos\theta - y\sin\theta \\ y' = x\sin\theta + y\cos\theta \end{cases}$

```
Point<ld> rotate(Point<ld> p, ld rad) {
    return {p.x * cos(rad) - p.y * sin(rad), p.x * sin(rad) + p.y * cos(rad)};
}
```

点绕点旋转任意角度

逆时针旋转，转换公式：$\begin{cases} x' = (x_0 - x_1)\cos\theta + (y_0 - y_1)\sin\theta + x_1 \\ y' = (x_1 - x_0)\sin\theta + (y_0 - y_1)\cos\theta + y_1 \end{cases}$

```
Point<ld> rotate(Point<ld> a, Point<ld> b, ld rad) {
    ld x = (a.x - b.x) * cos(rad) + (a.y - b.y) * sin(rad) + b.x;
    ld y = (b.x - a.x) * sin(rad) + (a.y - b.y) * cos(rad) + b.y;
    return {x, y};
}
```

平面点线相关

点是否在直线上（三点是否共线）

```
template<class T> bool onLine(Point<T> a, Point<T> b, Point<T> c) {
    return sign(cross(b, a, c)) == 0;
}
template<class T> bool onLine(Point<T> p, Line<T> l) {
    return onLine(p, l.a, l.b);
}
```

点是否在向量（直线）左侧

需要注意，向量的方向会影响答案；点在向量上时不视为在左侧。

```
template<class T> bool pointOnLineLeft(Pt p, Lt l) {
    return cross(l.b, p, l.a) > 0;
}
```

两点是否在直线同侧/异侧

```
template<class T> bool pointOnLineSide(Pt p1, Pt p2, Lt vec) {
    T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
    return sign(val) == 1;
}
template<class T> bool pointNotOnLineSide(Pt p1, Pt p2, Lt vec) {
    T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);
    return sign(val) == -1;
}
```

两直线相交交点

在使用前需要先判断直线是否平行。

```
Pd lineIntersection(Ld l1, Ld l2) {
    ld val = cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b);
    return l1.a + (l1.b - l1.a) * val;
}
```

两直线是否平行/垂直/相同

```
template<class T> bool lineParallel(Lt p1, Lt p2) {
    return sign(cross(p1.a - p1.b, p2.a - p2.b)) == 0;
}
template<class T> bool lineVertical(Lt p1, Lt p2) {
    return sign(dot(p1.a - p1.b, p2.a - p2.b)) == 0;
}
template<class T> bool same(Line<T> l1, Line<T> l2) {
    return lineParallel(Line{l1.a, l2.b}, {l1.b, l2.a}) &&
           lineParallel(Line{l1.a, l2.a}, {l1.b, l2.b}) && lineParallel(l1, l2);
}
```

点到直线的最近距离与最近点

```
pair<Pd, ld> pointToLine(Pd p, Ld l) {
    Pd ans = lineIntersection({p, p + rotate(l.a, l.b)}, l);
    return {ans, dis(p, ans)};
}
```

如果只需要计算最近距离，下方的写法可以减少书写的代码量，效果一致。

```
template<class T> ld disPointToLine(Pt p, Lt l) {
    ld ans = cross(p, l.a, l.b);
    return abs(ans) / dis(l.a, l.b); // 面积除以底边长
}
```

点是否在线段上

```
template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点也算作在直线上
    return sign(cross(p, l.a, l.b)) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <=
max(l.a.x, l.b.x) &&
           min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}
template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点不算
    return pointOnSegment(p, l) && min(l.a.x, l.b.x) < p.x && p.x < max(l.a.x,
l.b.x) &&
           min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b.y);
}
```

点到线段的最近距离与最近点

```
pair<Pd, ld> pointToSegment(Pd p, Ld l) {
    if (sign(dot(p, l.b, l.a)) == -1) { // 特判到两端点的距离
        return {l.a, dis(p, l.a)};
    } else if (sign(dot(p, l.a, l.b)) == -1) {
        return {l.b, dis(p, l.b)};
    }
    return pointToLine(p, l);
}
```

点在直线上的投影点（垂足）

```
Pd project(Pd p, Ld l) { // 投影
    Pd vec = l.b - l.a;
    ld r = dot(vec, p - l.a) / (vec.x * vec.x + vec.y * vec.y);
    return l.a + vec * r;
}
```

线段的中垂线

```
template<class T> Lt midSegment(Lt l) {
    Pt mid = (l.a + l.b) / 2; // 线段中点
    return {mid, mid + rotate(l.a, l.b)};
}
```

两线段是否相交及交点

该扩展版可以同时返回相交状态和交点，分为四种情况：0 代表不相交；1 代表普通相交；2 代表重叠（交于两个点）；3 代表相交于端点。**需要注意**，部分运算可能会使用到直线求交点，此时务必保证变量类型为浮点数！

```
template<class T> tuple<int, Pt, Pt> segmentIntersection(Lt l1, Lt l2) {
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
    auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
    auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
    auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
    if (A < CC || C < AA || B < DD || D < BB) {
        return {0, {}, {}};
    }
    if (sign(cross(e1 - s1, e2 - s2)) == 0) {
        if (sign(cross(s2, e1, s1)) != 0) {
            return {0, {}, {}};
        }
        Pt p1(max(AA, CC), max(BB, DD));
        Pt p2(min(A, C), min(B, D));
        if (!pointOnSegment(p1, l1)) {
            swap(p1.y, p2.y);
        }
        if (p1 == p2) {
            return {3, p1, p2};
        } else {
            return {2, p1, p2};
        }
    }
    auto cp1 = cross(s2 - s1, e2 - s1);
    auto cp2 = cross(s2 - e1, e2 - e1);
    auto cp3 = cross(s1 - s2, e1 - s2);
    auto cp4 = cross(s1 - e2, e1 - e2);
    if (sign(cp1 * cp2) == 1 || sign(cp3 * cp4) == 1) {
        return {0, {}, {}};
    }
    // 使用下方函数时请使用浮点数
    Pd p = lineIntersection(l1, l2);
    if (sign(cp1) != 0 && sign(cp2) != 0 && sign(cp3) != 0 && sign(cp4) != 0) {
        return {1, p, p};
    } else {
        return {3, p, p};
    }
}
```

如果不需要求交点，那么使用快速排斥+跨立实验即可，其中重叠、相交于端点均视为不相交。

```
template<class T> bool segmentIntersection(Lt l1, Lt l2) {
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
    auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
    auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
    auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
    return A >= CC && B >= DD && C >= AA && D >= BB &&
           sign(cross(s1, s2, e1) * cross(s1, e1, e2)) == 1 &&
           sign(cross(s2, s1, e2) * cross(s2, e2, e1)) == 1;
}//其中重叠、相交于端点均视为不相交。
```

```
template<class T> bool segmentIntersection(Line<T> l1, Line<T> l2) {
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
    auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
    auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
    auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
    if (A < CC || B < DD || C < AA || D < BB) return false;
    T c1 = cross(e1 - s1, s2 - s1), c2 = cross(e1 - s1, e2 - s1);
    T c3 = cross(e2 - s2, s1 - s2), c4 = cross(e2 - s2, e1 - s2);
    if (sign(c1) == 0 && sign(c2) == 0) {
        // 共线，快速排斥实验已经通过，则相交
        return true;
    }
    return sign(c1) * sign(c2) <= 0 && sign(c3) * sign(c4) <= 0;
}//其中重叠、相交于端点均视为相交。
```

线段之间距离

```
ld DisSegToSeg(Ld l1,Ld l2)
{
    // if(lineParallel(l1,l2))
    // {
    //     auto [t1,ans1]=pointToSegment(l1.a,l2);
    //     return ans1;
    // }
    if(segmentIntersection(l1,l2)) return 0.0;
    auto [t1,ans1]=pointToSegment(l1.a,l2);
    auto [t2,ans2]=pointToSegment(l1.b,l2);
    auto [t3,ans3]=pointToSegment(l2.a,l1);
    auto [t4,ans4]=pointToSegment(l2.b,l1);
    //cout<<ans1<<' '<<ans2<<' '<<ans3<<' '<<ans4<<'\n';
    return min(min(ans1,ans2),min(ans3,ans4));
}
```

平面圆相关（浮点数处理）

点到圆的最近点

同时返回最近点与最近距离。**需要注意**，当点为圆心时，这样的点有无数个，此时我们视作输入错误，直接返回圆心。

```
pair<Pd, ld> pointToCircle(Pd p, Pd o, ld r) {
    Pd U = o, V = o;
    ld d = dis(p, o);
    if (sign(d) == 0) { // p 为圆心时返回圆心本身
        return {o, 0};
    }
    ld val1 = r * abs(o.x - p.x) / d;
    ld val2 = r * abs(o.y - p.y) / d * ((o.x - p.x) * (o.y - p.y) < 0 ? -1 : 1);
    U.x += val1, U.y += val2;
    V.x -= val1, V.y -= val2;
    if (dis(U, p) < dis(V, p)) {
        return {U, dis(U, p)};
    } else {
        return {V, dis(V, p)};
    }
}
```

根据圆心角获取圆上某点

将圆上最右侧的点以圆心为旋转中心，逆时针旋转 `rad` 度。

```
Point<ld> getPoint(Point<ld> p, ld r, ld rad) {
    return {p.x + cos(rad) * r, p.y + sin(rad) * r};
}
```

直线是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交。

```
tuple<int, Pd, Pd> lineCircleCross(Ld l, Pd o, ld r) {
    Pd P = project(o, l);
    ld d = dis(P, o), tmp = r * r - d * d;
    if (sign(tmp) == -1) {
        return {0, {}, {}};
    } else if (sign(tmp) == 0) {
        return {1, P, {}};
    }
    Pd vec = standardize(l.b - l.a) * sqrt(tmp);
    return {2, P + vec, P - vec};
}
```

线段是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交于一个点；3 代表相交于两个点。

```
tuple<int, Pd, Pd> segmentCircleCross(Ld l, Pd o, ld r) {
    auto [type, U, V] = lineCircleCross(l, o, r);
    bool f1 = pointOnSegment(U, l), f2 = pointOnSegment(V, l);
    if (type == 1 && f1) {
        return {1, U, {}};
    } else if (type == 2 && f1 && f2) {
        return {3, U, V};
    } else if (type == 2 && f1) {
        return {2, U, {}};
    } else if (type == 2 && f2) {
        return {2, V, {}};
    } else {
        return {0, {}, {}};
    }
}
```

两圆是否相交及交点

0 代表内含；1 代表相离；2 代表相切；3 代表相交。

```
tuple<int, Pd, Pd> circleIntersection(Pd p1, ld r1, Pd p2, ld r2) {
    ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
    if (sign(abs(r1 - r2) - d) == 1) {
        return {0, {}, {}};
    } else if (sign(r1 + r2 - d) == -1) {
        return {1, {}, {}};
    }
    ld a = r1 * (x1 - x2) * 2, b = r1 * (y1 - y2) * 2, c = r2 * r2 - r1 * r1 - d *
d;
    ld p = a * a + b * b, q = -a * c * 2, r = c * c - b * b;
    ld cosa, sina, cosb, sinb;
    if (sign(d - (r1 + r2)) == 0 || sign(d - abs(r1 - r2)) == 0) {
        cosa = -q / p / 2;
        sina = sqrt(1 - cosa * cosa);
        Point<ld> p0 = {x1 + r1 * cosa, y1 + r1 * sina};
        if (sign(dis(p0, p2) - r2)) {
            p0.y = y1 - r1 * sina;
        }
        return {2, p0, p0};
    } else {
        ld delta = sqrt(q * q - p * r * 4);
        cosa = (delta - q) / p / 2;
        cosb = (-delta - q) / p / 2;
        sina = sqrt(1 - cosa * cosa);
        sinb = sqrt(1 - cosb * cosb);
        Pd ans1 = {x1 + r1 * cosa, y1 + r1 * sina};
        Pd ans2 = {x1 + r1 * cosb, y1 + r1 * sinb};
```

```
        if (sign(dis(ans1, p1) - r2)) ans1.y = y1 - r1 * sina;
        if (sign(dis(ans2, p2) - r2)) ans2.y = y1 - r1 * sinb;
        if (ans1 == ans2) ans1.y = y1 - r1 * sina;
        return {3, ans1, ans2};
    }
}
```

修正

```
tuple<int, Pd, Pd> circleIntersection(Pd p1, ld r1, Pd p2, ld r2) {
    ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y;
    ld d = dis(p1, p2);

    // 检查圆的位置关系
    if (sign(d) == 0) { // 同心圆
        if (sign(r1 - r2) == 0) return {4, {}, {}}; // 重合
        else return {0, {}, {}}; // 内含
    }

    if (sign(d - (r1 + r2)) == 1) { // 相离
        return {0, {}, {}};
    } else if (sign(abs(r1 - r2) - d) == 1) { // 内含
        return {0, {}, {}};
    }

    // 计算交点
    ld dx = x2 - x1, dy = y2 - y1;
    ld a = (r1*r1 - r2*r2 + d*d) / (2*d);
    ld h = sqrt(r1*r1 - a*a);

    // 中点
    ld xm = x1 + a * dx / d;
    ld ym = y1 + a * dy / d;

    // 垂直方向的偏移量
    ld xoff = -h * dy / d;
    ld yoff = h * dx / d;

    Pd ans1 = {xm + xoff, ym + yoff};
    Pd ans2 = {xm - xoff, ym - yoff};

    if (sign(dis(ans1, ans2)) == 0) { // 相切，一个交点
        return {2, ans1, ans1};
    } else { // 相交，两个交点
        return {3, ans1, ans2};
    }
}
```

扇形面积

```
ld getSector(Pd o,ld r,Pd A,Pd B)
{
    auto an=angle(o-A,o-B);
    return r*r*an*0.5;
}
```

两圆相交面积

上述所言四种相交情况均可计算，之所以不使用三角形面积计算公式是因为在计算过程中会出现“负数”面积（扇形面积与三角形面积的符号关系会随圆的位置关系发生变化），故公式全部重新推导，这里采用的是扇形面积减去扇形内部的那个三角形的面积。

```
ld circleIntersectionArea(Pd p1, ld r1, Pd p2, ld r2) {
    ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
    if (sign(abs(r1 - r2) - d) >= 0) {
        return PI * min(r1 * r1, r2 * r2);
    } else if (sign(r1 + r2 - d) == -1) {
        return 0;
    }
    ld theta1 = angle(r1, dis(p1, p2), r2);
    ld area1 = r1 * r1 * (theta1 - sin(theta1 * 2) / 2);
    ld theta2 = angle(r2, dis(p1, p2), r1);
    ld area2 = r2 * r2 * (theta2 - sin(theta2 * 2) / 2);
    return area1 + area2;
}
```

三点确定一圆

```
tuple<int, Pd, ld> getCircle(Pd A, Pd B, Pd C) {
    if (onLine(A, B, C)) { // 特判三点共线
        return {0, {}, 0};
    }
    Ld l1 = midSegment(Line{A, B});
    Ld l2 = midSegment(Line{A, C});
    Pd O = lineIntersection(l1, l2);
    return {1, O, dis(A, O)};
}
```

点集中最小覆盖圆

```
// 计算最小包围圆（最小圆覆盖）
// 输入：点集的迭代器范围 [left, right)，随机种子 seed
// 输出：pair<圆心，半径平方>
template <class iter>
std::pair<P, ld> min_ball(iter left, iter right, int seed = 1333) {
    const int n = right - left;  // 点的数量

    assert(n >= 1);  // 至少需要1个点
```

```
    // 特殊情况处理：只有1个点
    if (n == 1) {
        return {*left, ld(0)};  // 圆就是点本身，半径为0
    }

    // 随机数生成器
    std::mt19937 mt(seed);
    // 随机打乱点集顺序 - 这是保证O(n)期望时间复杂度的关键
    std::shuffle(left, right, mt);

    iter ps = left;  // 打乱后的点集起始位置
    using circle = std::pair<P, ld>;  // 圆类型：圆心 + 半径平方

    // 通过三个点构造圆的lambda函数
    auto make_circle_3 = [](const P &a, const P &b, const P &c) -> circle {
        // 使用三角形外心公式计算圆心
        ld A = std::norm(b - c),  // |BC|²
           B = std::norm(c - a),  // |CA|²
           C = std::norm(a - b),  // |AB|²
           S = cross(b - a, c - a);  // 三角形面积的2倍

        // 外心坐标公式（重心坐标形式）
        P p = (A * (B + C - A) * a + B * (C + A - B) * b + C * (A + B - C) * c) / (4
* S * S);
        ld r2 = std::norm(p - a);  // 半径的平方
        return {p, r2};
    };

    // 通过两个点构造圆的lambda函数（以两点为直径的圆）
    auto make_circle_2 = [](const P &a, const P &b) -> circle {
        P c = (a + b) / (ld)2;  // 圆心为两点中点
        ld r2 = std::norm(a - c);  // 半径平方
        return {c, r2};
    };

    // 判断点是否在圆内的lambda函数
    auto in_circle = [](const P &a, const circle &c) -> bool {
        // 判断点到圆心的距离平方是否 <= 半径平方 + 容误差
        return std::norm(a - c.first) <= c.second + eps;
    };

    // 初始化：用前两个点构造初始圆
    circle c = make_circle_2(ps[0], ps[1]);

    // 主循环：逐个处理剩余的点
    for (int i = 2; i < n; ++i) {
        // 如果当前点不在当前圆内，需要更新圆
        if (!in_circle(ps[i], c)) {
            // MiniDiscWithPoint: 新圆必须包含当前点ps[i]
            c = make_circle_2(ps[0], ps[i]);  // 初始尝试：用第一个点和当前点构造圆
```

```
            // 检查前面的点是否都在新圆内
            for (int j = 1; j < i; ++j) {
                if (!in_circle(ps[j], c)) {
                    // MiniDiscWith2Points: 新圆必须包含ps[i]和ps[j]
                    c = make_circle_2(ps[i], ps[j]);  // 用当前两个点构造圆

                    // 检查更前面的点
                    for (int k = 0; k < j; ++k) {
                        if (!in_circle(ps[k], c)) {
                            // 三个点确定唯一圆
                            c = make_circle_3(ps[i], ps[j], ps[k]);
                        }
                    }
                }
            }
        }
        // 如果当前点在圆内，直接跳过（概率O(1/k)，这是算法高效的关键）
    }
    return c;
}
```

求解点到圆的切线数量与切点

```
pair<int, vector<Point<ld>>> tangent(Point<ld> p, Point<ld> A, ld r) {
    vector<Point<ld>> ans; // 储存切点
    Point<ld> u = A - p;
    ld d = sqrt(dot(u, u));
    if (d < r) {
        return {0, {}};
    } else if (sign(d - r) == 0) { // 点在圆上
        ans.push_back(u);
        return {1, ans};
    } else {
        ld ang = asin(r / d);
        ans.push_back(getPoint(A, r, -ang));
        ans.push_back(getPoint(A, r, ang));
        return {2, ans};
    }
}
```

修正

```
pair<int, vector<Point<ld>>> tangent(Point<ld> p, Point<ld> A, ld r) {
    vector<Point<ld>> ans;
    ld d = dis(p, A);

    if (sign(d - r) < 0) {
        // 点在圆内，无切线
        return {0, {}};
```

```
    } else if (sign(d - r) == 0) {
        // 点在圆上，只有一条切线
        // 切点就是p本身
        ans.push_back(p);
        return {1, ans};
    } else {
        // 点在圆外，有两条切线
        Point<ld> u = A - p;

        // 使用圆心和半径计算
        // 计算圆心到切点的方向角
        ld alpha = atan2(u.y, u.x);
        ld beta = asin(r / d);

        Point<ld> t1_alt = getPoint(A, r, alpha + beta + M_PI_2);
        Point<ld> t2_alt = getPoint(A, r, alpha - beta - M_PI_2);

        ans.push_back(t1_alt);
        ans.push_back(t2_alt);

        return {2, ans};
    }
}
```

求解两圆的内公、外公切线数量与切点

同时返回公切线数量以及每个圆的切点。

```
tuple<int, vector<Point<ld>>, vector<Point<ld>>> tangent(Point<ld> A, ld Ar,
Point<ld> B, ld Br) {
    vector<Point<ld>> a, b; // 储存切点
    if (Ar < Br) {
        swap(Ar, Br);
        swap(A, B);
        swap(a, b);
    }
    int d = disEx(A, B), dif = Ar - Br, sum = Ar + Br;
    if (d < dif * dif) { // 内含，无
        return {0, {}, {}};
    }
    ld base = atan2(B.y - A.y, B.x - A.x);
    if (d == 0 && Ar == Br) { // 完全重合，无数条外公切线
        return {-1, {}, {}};
    }
    if (d == dif * dif) { // 内切，1条外公切线
        a.push_back(getPoint(A, Ar, base));
        b.push_back(getPoint(B, Br, base));
        return {1, a, b};
    }
    ld ang = acos(dif / sqrt(d));
    a.push_back(getPoint(A, Ar, base + ang)); // 保底2条外公切线
    a.push_back(getPoint(A, Ar, base - ang));
```

```
    b.push_back(getPoint(B, Br, base + ang));
    b.push_back(getPoint(B, Br, base - ang));
    if (d == sum * sum) { // 外切，多1条内公切线
        a.push_back(getPoint(A, Ar, base));
        b.push_back(getPoint(B, Br, base + PI));
    } else if (d > sum * sum) { // 相离，多2条内公切线
        ang = acos(sum / sqrt(d));
        a.push_back(getPoint(A, Ar, base + ang));
        a.push_back(getPoint(A, Ar, base - ang));
        b.push_back(getPoint(B, Br, base + ang + PI));
        b.push_back(getPoint(B, Br, base - ang + PI));
    }
    return {a.size(), a, b};
}
```

最小包含圆

```
// 计算最小包围圆（最小圆覆盖）
// 输入：点集的迭代器范围 [left, right)，随机种子 seed
// 输出：pair<圆心, 半径平方>
template <class T>
std::pair<Point<T>, ld> min_ball(vector<Pt> A, int seed = 1333) {

    const int n = A.size();  // 点的数量

    assert(n >= 1);  // 至少需要1个点

    // 特殊情况处理：只有1个点
    if (n == 1) {
        return {A[0], ld(0)};  // 圆就是点本身，半径为0
    }

    // 随机数生成器
    std::mt19937 mt(seed);
    // 随机打乱点集顺序 - 这是保证O(n)期望时间复杂度的关键
    std::shuffle(A.begin(), A.end(), mt);

    vector<Pt> &ps=A;

    using circle = std::pair<Pt, ld>;  // 圆类型：圆心 + 半径平方

    // 通过三个点构造圆的lambda函数
    auto make_circle_3 = [](const Pt &a, const Pt &b, const Pt &c) -> circle {
        // 使用三角形外心公式计算圆心
        ld A = disEx(b , c),  // |BC|²
           B = disEx(c , a),  // |CA|²
           C = disEx(a , b),  // |AB|²
           S = cross(b - a, c - a);  // 三角形面积的2倍

        // 外心坐标公式（重心坐标形式）
```

```
        Pt p = (A * (B + C - A) * a + B * (C + A - B) * b + C * (A + B - C) * c) /
(4 * S * S);
        ld r2 = disEx(p , a);  // 半径的平方
        return {p, r2};
    };

    // 通过两个点构造圆的lambda函数（以两点为直径的圆）
    auto make_circle_2 = [](const Pt &a, const Pt &b) -> circle {
        Pt c = (a + b) / (ld)2;  // 圆心为两点中点
        ld r2 = disEx(a , c);  // 半径平方
        return {c, r2};
    };

    // 判断点是否在圆内的lambda函数
    auto in_circle = [](const Pt &a, const circle &c) -> bool {
        // 判断点到圆心的距离平方是否 <= 半径平方 + 容误差
        return disEx(a , c.first) <= c.second + EPS;
    };

    // 初始化：用前两个点构造初始圆
    circle c = make_circle_2(ps[0], ps[1]);

    // 主循环：逐个处理剩余的点
    for (int i = 2; i < n; ++i) {
        // 如果当前点不在当前圆内，需要更新圆
        if (!in_circle(ps[i], c)) {
            // MiniDiscWithPoint: 新圆必须包含当前点ps[i]
            c = make_circle_2(ps[0], ps[i]);  // 初始尝试：用第一个点和当前点构造圆

            // 检查前面的点是否都在新圆内
            for (int j = 1; j < i; ++j) {
                if (!in_circle(ps[j], c)) {
                    // MiniDiscWith2Points: 新圆必须包含ps[i]和ps[j]
                    c = make_circle_2(ps[i], ps[j]);  // 用当前两个点构造圆

                    // 检查更前面的点
                    for (int k = 0; k < j; ++k) {
                        if (!in_circle(ps[k], c)) {
                            // 三个点确定唯一圆
                            c = make_circle_3(ps[i], ps[j], ps[k]);
                        }
                    }
                }
            }
        }
        // 如果当前点在圆内，直接跳过（概率O(1/k)，这是算法高效的关键）
    }
    return c;
}
```

平面三角形相关（浮点数处理）

三角形面积

```
ld area(Point<ld> a, Point<ld> b, Point<ld> c) {
    return abs(cross(b, c, a)) / 2;
}
```

三角形外心

三角形外接圆的圆心，即三角形三边垂直平分线的交点。

```
template<class T> Pt center1(Pt p1, Pt p2, Pt p3) { // 外心
    return lineIntersection(midSegment({p1, p2}), midSegment({p2, p3}));
}
```

三角形内心

三角形内切圆的圆心，也是三角形三个内角的角平分线的交点。其到三角形三边的距离相等。

```
Pd center2(Pd p1, Pd p2, Pd p3) { // 内心
    #define atan2(p) atan2(p.y, p.x) // 注意先后顺序
    Line<ld> U = {p1, {}}, V = {p2, {}};
    ld m, n, alpha;
    m = atan2((p2 - p1));
    n = atan2((p3 - p1));
    alpha = (m + n) / 2;
    U.b = {p1.x + cos(alpha), p1.y + sin(alpha)};
    m = atan2((p1 - p2));
    n = atan2((p3 - p2));
    alpha = (m + n) / 2;
    V.b = {p2.x + cos(alpha), p2.y + sin(alpha)};
    return lineIntersection(U, V);
}
```

三角形内接圆

```
pair<Pd,ld> getIncircle(Pd A,Pd B,Pd C)
{
    ld AB=dis(A,B);
    ld AC=dis(A,C);
    ld BC=dis(B,C);
    Pd ans((AB*C.x+AC*B.x+BC*A.x)/(AB+AC+BC),(AB*C.y+AC*B.y+BC*A.y)/(AB+AC+BC));
    ld ans1=cross(A,B,C)/(AB+AC+BC);
    return {ans,max(ans1,-ans1)};
}
```

三角形垂心

三角形的三条高线所在直线的交点。锐角三角形的垂心在三角形内；直角三角形的垂心在直角顶点上；钝角三角形的垂心在三角形外。

```
Pd center3(Pd p1, Pd p2, Pd p3) { // 垂心
    Ld U = {p1, p1 + rotate(p2, p3)}; // 垂线
    Ld V = {p2, p2 + rotate(p1, p3)};
    return lineIntersection(U, V);
}
```

平面直线方程转换

浮点数计算直线的斜率

一般很少使用到这个函数，因为斜率的取值不可控（例如接近平行于 x, y 轴时）。**需要注意**，当直线平行于 y 轴时斜率为 `inf`。

```
template <class T> ld slope(Pt p1, Pt p2) { // 斜率，注意 inf 的情况
    return (p1.y - p2.y) / (p1.x - p2.x);
}
template <class T> ld slope(Lt l) {
    return slope(l.a, l.b);
}
```

分数精确计算直线的斜率

调用分数四则运算精确计算斜率，返回最简分数，只适用于整数计算。

```
template<class T> Frac<T> slopeEx(Pt p1, Pt p2) {
    Frac<T> U = p1.y - p2.y;
    Frac<T> V = p1.x - p2.x;
    return U / V; // 调用分数精确计算
}
```

两点式转一般式

返回由三个整数构成的方程，在输入较大时可能找不到较小的满足题意的一组整数解。可以处理平行于 x, y 轴、两点共点的情况。

```
template<class T> tuple<int, int, int> getfun(Lt p) {
    T A = p.a.y - p.b.y, B = p.b.x - p.a.x, C = p.a.x * A + p.a.y * B;
    if (A < 0) { // 符号调整
        A = -A, B = -B, C = -C;
    } else if (A == 0) {
        if (B < 0) {
            B = -B, C = -C;
        } else if (B == 0 && C < 0) {
            C = -C;
        }
```

```
    }
    if (A == 0) { // 数值计算
        if (B == 0) {
            C = 0; // 共点特判
        } else {
            T g = fgcd(abs(B), abs(C));
            B /= g, C /= g;
        }
    } else if (B == 0) {
        T g = fgcd(abs(A), abs(C));
        A /= g, C /= g;
    } else {
        T g = fgcd(fgcd(abs(A), abs(B)), abs(C));
        A /= g, B /= g, C /= g;
    }
    return tuple{A, B, C}; // Ax + By = C
}
```

一般式转两点式

由于整数点可能很大或者不存在，故直接采用浮点数；如果与 x, y 轴有交点则取交点。可以处理平行于 x, y 轴的情况。

```
Line<ld> getfun(int A, int B, int C) { // Ax + By = C
    ld x1 = 0, y1 = 0, x2 = 0, y2 = 0;
    if (A && B) { // 正常
        if (C) {
            x1 = 0, y1 = 1. * C / B;
            y2 = 0, x2 = 1. * C / A;
        } else { // 过原点
            x1 = 1, y1 = 1. * -A / B;
            x2 = 0, y2 = 0;
        }
    } else if (A && !B) { // 垂直
        if (C) {
            y1 = 0, x1 = 1. * C / A;
            y2 = 1, x2 = x1;
        } else {
            x1 = 0, y1 = 1;
            x2 = 0, y2 = 0;
        }
    } else if (!A && B) { // 水平
        if (C) {
            x1 = 0, y1 = 1. * C / B;
            x2 = 1, y2 = y1;
        } else {
            x1 = 1, y1 = 0;
            x2 = 0, y2 = 0;
        }
    } else { // 不合法，请特判
        assert(false);
```

```
    }
    return {{x1, y1}, {x2, y2}};
}
```

抛物线与 x 轴是否相交及交点

0 代表没有交点；1 代表相切；2 代表有两个交点。

```
tuple<int, ld, ld> getAns(ld a, ld b, ld c) {
    ld delta = b * b - a * c * 4;
    if (delta < 0.) {
        return {0, 0, 0};
    }
    delta = sqrt(delta);
    ld ans1 = -(delta + b) / 2 / a;
    ld ans2 = (delta - b) / 2 / a;
    if (ans1 > ans2) {
        swap(ans1, ans2);
    }
    if (sign(delta) == 0) {
        return {1, ans2, 0};
    }
    return {2, ans1, ans2};
}
```

在计算几何中，以原点 $(0,0)$ 为极点的极角排序有两种常见方法：

1. **使用** `atan2(y, x)`： 转化为浮点数计算极角。缺点是精度容易丢失，在坐标范围达到 10^9 时极易出错。
2. **使用 半平面划分 + 叉积： （强烈推荐）** 完全使用整数运算，0 精度误差。

基于你提供的板子（特别是你已经有了完整的 `cross` 和距离函数 `disEx`），我为你实现**第二种无精度误差**的极角排序。

核心思想：

极角排序通常从正 X 轴开始，逆时针转一圈。我们可以把平面分成上下两半：

1. **上半平面**（包括正 X 轴）：极角在 $[0, \pi)$ 之间。
2. **下半平面**（包括负 X 轴）：极角在 $[\pi, 2\pi)$ 之间。

排序规则：

1. 如果两个点不在同一个半平面，上半平面的点排在前面。
2. 如果两个点在同一个半平面，用 **叉积** `cross(a, b)` 判断先后顺序。如果 `cross(a, b) > 0`（说明 a 到 b 是左转/逆时针），则 a 排在前面。
3. 如果叉积为 0（说明极角相同/共线），按到原点的**距离**从小到大排序。

具体实现代码

你可以直接在你的代码中加入以下部分（假设你使用了 `using ll = long long;` 和 `using Pt = Point<ll>;`）：

```
// 确保你的 Pt 别名已经定义，例如：
// using ll = long long;
// using Pt = Point<ll>;

// 极角排序的比较函数
template<class T> bool polar_cmp(const Pt& a, const Pt& b) {
    // 辅助函数：判断点在哪个半平面
    // 返回 0 表示上半平面（包括正 X 轴，y > 0 或者 y == 0 且 x > 0）
    // 返回 1 表示下半平面（包括负 X 轴，y < 0 或者 y == 0 且 x < 0）
    auto get_half = [](const Pt& p) {
        if (p.y > 0 || (p.y == 0 && p.x > 0)) return 0;
        return 1;
    };

    int ha = get_half(a);
    int hb = get_half(b);

    // 1. 如果不在同一个半平面，上半平面的优先
    if (ha != hb) {
        return ha < hb;
    }

    // 2. 如果在同一半平面，用叉积判断角度
    // cross(a, b) > 0 说明 a 的极角小于 b 的极角
    ll c = cross(a, b);
    if (c != 0) {
        return c > 0;
    }

    // 3. 极角相同时（共线），按距离原点的距离从小到大排
    // 这里使用了你板子里的 disEx (距离的平方)
    Pt origin(0, 0);
    return disEx<ll>(a, origin) < disEx<ll>(b, origin);
}
```

如何使用?

在主函数中，当你读入除了原点 O 之外的其他点存入 `vector<Pt> pts` 后，直接调用 `std::sort` 即可：

```
// 假设 pts 是去除了原点 O(0,0) 的点集
sort(pts.begin(), pts.end(), polar_cmp);
```

###